Pokročilé informační systémy

Business vrtstva a aplikační rozhraní

doc. Ing. Radek Burget, Ph.D.
burgetr@fit.vutbr.cz

Implementace business operací

- Implementace business logiky nezávisle na prezentační vrstvě
- Opakovaně použitelné metody
  - Propojení s ostatními vrstvami
- Správa transakcí
- Příp. distribuce na aplikační servery

Enterprise Java Beans (EJB)

- Enterprise Java Beans (EJB)
  - Zapouzdřují business logiku aplikace
  - Poskytují business operace – definované rozhraní (metody)
  - EJB kontejner zajišťuje další služby
    - Dependency injection
    - Transakční zpracování
      - Metoda obvykle tvoří transakci, není-li nastaveno jinak

Vytvoření EJB

- Instance vytváří a spravuje EJB kontejner
- Vytvoření pomocí anotace třídy
  - `@Stateless` – bezstavový bean
    - Efektivnější správa – pool objektů přidělovaných klientům
  - `@Stateful` – udržuje se stav
    - Jedna instance na klienta
  - `@Singleton`
    - Jedna instance na celou aplikaci

Použití EJB

- Lokální
  - Anotace `@EJB` – kontejner dodá instanci EJB
- Vzdálené volání – dané rozhraní
  - Rozhraní definované pomocí `@Remote`

Contexts and Dependency Injection (CDI)

- Obecný mechanismus pro DI mimo EJB
- Omezuje závislosti mezi třídami přímo v kódu
  - Flexibilita (výměna implementace), lepší testování, …
- Injektovatelné objekty
  - Třídy, které nejsou EJB
  - Různé vlastnosti pomocí anotací
- Použití objektu
  - Anotace `@Inject`
  - CDI kontejner zajistí získáni a dodání instance

CDI – Injektovatelné objekty

- Téměř jakákoliv Javovská třída
- Scope
  - `@Dependent` – vzniká pro konkrétní případ, zaniká s vlastníkem (default)
  - `@RequestScoped` – trvá po dobu HTTP požadavku
  - `@SessionScoped` – trvá po dobu HTTP session
  - `@ApplicationScoped` – jedna instance pro aplikaci
  - *Pozor na shodu jmen se staršími anotacemi JSF*
- Pokud má být přístupný z GUI (pomocí EL)
  - Anotace `@Named`

CDI – Dodání instancí

- Anotace `@Inject`
- Vlastnost (field)

```
@WebServlet(urlPatterns = "/itemServlet")
public class ItemServlet extends HttpServlet {

    @Inject
    private NumberGenerator numberGenerator;

}
```

CDI – Dodání instancí

- Konstruktor

```
@WebServlet(urlPatterns = "/itemServlet")
public class ItemServlet extends HttpServlet {
    private NumberGenerator numberGenerator;

    @Inject
    public ItemServlet(NumberGenerator numberGenerator) {
        this.numberGenerator = numberGenerator;
    }

    …
}
```

CDI – Dodání instancí

- Setter

```
@WebServlet(urlPatterns = "/itemServlet")
public class ItemServlet extends HttpServlet {

    private NumberGenerator numberGenerator;

    @Inject
    public void setNumberGenerator(NumberGenerator numberGenerator)   {
        this.numberGenerator = numberGenerator;
    }

    …
}
```

Webové API

REST rozhraní pomocí JAX-RS

REST

- Předpokládá CRUD (Create-Retrieve-Update-Delete) operace s *entitami*
  - Ale ve skutečnosti přistupujeme k **business vrstvě**, ne přímo k datům!
  - Tzn. **voláme aplikační logiku**
- Úzká vazba na HTTP
  - Využití HTTP metod a jejich významu
  - Využití stavových kódů v HTTP
- Nedefinuje formát přenosu dat, obvykle JSON, méně XML (často obojí)

Endpointy

- Endpoint = URL, na které lze zaslat požadavek
- Reprezentuje **zdroj** (*resource*), který má nějaký **stav** (*state*)
- Endpointy pro operace se zdroji
  - Kolekce entit, např. `http://obchod.cz/api/objednavky`
  - Jedna entita, např. `http://obchod.cz/api/objednavky/8235`
- Endpointy pro volání funkcí
  - Např. `http://obchod.cz/api/odesli-objednavku`

Metody HTTP – Operace se zdroji

- **GET**
  - Čtení stavu zdroje (read)
- **POST**
  - Přidání podřízeného zdroje (přidání do kolekce, create)
- **PUT**
  - Nahrazení zdroje novým stavem (update)
- **PATCH**
  - Nahrazení části zdroje (update)
- **DELETE**
  - Smazání zdroje (delete)

Metody HTTP – Volání funkcí

- **GET** i **POST**
  - Vykoná operaci vrátí výsledek (serializovaná data)
- Pokud je výsledkem operace nový zdroj, jeho URL se vrátí v hlavičce `Location`.

Stavový kód

- Stavový kód odpovědi HTTP může odpovídat výsledku operace
- Typicky například:
  - 200 Ok
  - 201 Created
  - 400 Bad request
  - 403 Forbidden
  - 404 Not found
  - 500 Internal server error

Formát přenosu dat

- Není specifikován, záleží na službě
  - Obvykle JSON nebo XML (schéma záleží na aplikaci)
- Často více formátu k dispozici
  - Např. http://noviny.cz/clanky.xml http://noviny.cz/clanky.json
  - Využití MIME pro rozlišení typu, HTTP content negotiation

REST a Jakarta EE

- JAX-RS API součástí standardu
- Vytvoření služeb pomocí anotací
- Aplikační server zajistí funkci endpointu (JAX-RS servlet)
  - Mapování URL a HTTP metod na Javovské objekty a metody
  - Serializace a deserializace JSON/XML na objekty
- Různé implementace
  - JAX-RS – Jersey (Glassfish), Apache Axis
  - Serializace – Jackson, gson, MOXy, …
  - Neřešíme - je vždy součástí aplikačního serveru

REST v Javě

```
import javax.ws.rs.GET;
import javax.ws.rs.Produces;
import javax.ws.rs.Path;

@Path("/users/{username}")
public class UserResource {

    @GET
    @Produces("text/xml")
    public User getUser(@PathParam("username") String userName) {
        // volání business operací (aplikační logika)
        return someUser;
```

- Demo: endpointy

Konfigurace

- Třída odvozená od `javax.ws.rs.core.Application`
- Konfigurace pomocí anotací
- Např. ApplicationConfig

Klientská aplikace

- Zasílání REST požadavků
  - Jednotlivé JS frameworky mají vlastní infrastrukturu
- Prezentační logika
  - Navigace
  - Přechody mezi stránkami
  - Výpisy chyb, apod.
- Viz např. jednoduchý JS klient

Návrh REST rozhraní

- Architektura REST je volná, umožňuje „chaoticky“ přidávat endpointy podle potřeby
- **Systematický návrh je nutný**
  - Pevné datové struktury (vycházející z doménového modelu)
    - Včetně reprezentace chybových stavů
  - Mapování business operací na endpointy (vycházející z případů použití)
- Ideálně formální popis rozhraní
  - Mnohem lepší než sdílená tabulka

Popis služeb v REST

- WADL (Web Application Description Language)
  - Založený na XML
  - Podpora v Javě – např. Payara
- OpenAPI https://swagger.io/specification/
  - Používá YAML (alternativně JSON)
  - Např. Payara/Liberty http://localhost:8080/openapi/
  - Generátory rozhraní třetích stran, např. https://github.com/OpenAPITools

Autentizace v REST

- Protokol REST je definován jako **bezstavový**
  - Požadavek musí obsahovat vše, žádné ukládání stavu na serveru
- To teoreticky vylučuje možnost použití sessions pro autentizaci
  - Technicky to ale možné je
  - Problém např. pro mobilní klienty
- Alternativy pro autentizaci:
  - **HTTP Basic** autentizace (nutné HTTPS)
  - Použití tokenu validovatelného na serveru – např. **JWT**
  - Složitější mechanismus, např. OAuth

HTTP Basic

- Standardní mechanismus HTTP, využívá speciální hlavičky
- Požadavek musí obsahovat hlavičku **Authorization**
  - Obsahuje jméno a heslo; nešifrované pouze kódované (base64)
    - **Je nutné použít HTTPS**
- Pro nesprávnou nebo chybějící autentizaci server vrací **401 Auhtorization Required**
  - V hlavičce `WWW-Autenticate` je identifikace oblasti přihlášení
  - Klient tedy zjistí, že je nutná autentizace pro tuto oblast

JSON Web Token (JWT)

- Řetězec složený ze 3 částí
  - Header (hlavička) – účel, použité algoritmy (JSON)
  - Payload (obsah) – JSON data obsahující id uživatele, jehopráva, expiraci apod.
  - Signature (podpis) – pro ověření, že token nebyl podvržen nebo změněn cestou
- Tyto tři části se kódují (base64) a spojí do jednoho řetězce
  - **xxxxx.yyyyy.zzzzz**

Použití JWT

- Klient kontaktuje **autentizační server** a dodá autentizační údaje
  - Stejný server, jaký poskytuje API, nebo i úplně jiný (např. Twitter)
- Autentizační server vygeneruje podepsaný JWT a vrátí klientovi
- Klient předá JWT při každém volání API
  - Nejčastěji opět v hlavičce:

    `Authorization: Bearer xxxxx.yyyyy.zzzzz`
  - API ověří platnost, role uživatele může být přímo v JWT

JWT v Javě

- Součástí standardu Microprofile
  - Ne přímo součást Jakarta EE
  - Ale dostupná na běžných serverech (Payara, Liberty, …)
- Lze snadno spojit s JAX-RS
- Demo: https://github.com/DIFS-Teaching/rest-auth

Generování tokenu

- Nastavení hodnot *claims*
  - Zejména `iss`, `upn` a `groups`
- Podpis privántím klíčem (RSA, ECDSA), nebo symetrická kryptografie (HMAC)
- Klient uloží token a posílá s každým požadavkem
- Postup generování klíčů [např. zde]

Autorizace pomocí JWT

- Ověření podpisu pomocí veřejného klíče
- Ověření vydavatele (`iss`)
- Použití uživatelského jména (`upn`) a rolí (`groups`) pro autorizaci.

Autorizace v Javě

- Zapnutí pro celou aplikaci + ověření role u endpointu

```
@ApplicationPath("resources")
@LoginConfig(authMethod = "MP-JWT")
@DeclareRoles({ "admin", "staff", "customer" })
public class JAXRSConfiguration extends Application {

}
```

```
@GET
@Path("/protected")
@RolesAllowed("admin")
public String getProtected() {
    return "ok";
}
```

Konfigurace JWT

- Pomocí Java properties, např. soubor `META-INF/microprofile-config.properties`

```
mp.jwt.verify.publickey.location=/publicKey.pem
mp.jwt.verify.issuer=fitdemo
```

(přepokládá veřejný klíč v `/publicKey.pem`)

GraphQL

Typovaná aplikační rozhraní

GraphQL

- https://graphql.org/
- Motivace: klient (klienti) potřebují v různých situacích různá data
  - Např. stránka „seznam osob“ vs. „detail osoby“
  - REST endpoint vrací vždy stejnou strukturu
    - Redundance dat (nevyužijeme všechna data)
    - Více dotazů ((ne)efektivita, složitější logika klienta)
- Získat některá data může být drahé (pokud je klient nepotřebuje)
- Řešení GraphQL
  - Popis datového modelu API
  - Dotaz na API specifikuje požadovaný tvar odpovědi
- **Předvídatelný výsledek dotazu**

GraphQL – datový model

- Datový model API (ne nutně serverové aplikace)
- Jednoduché datové typy: Int, Float, String, Boolean, ID, enum
- Uživatelské typy (*types*) = struktury
  - Vlastnosti (parametrizovatelné) – jméno, parametry, typ
  - Typy jednoduché, struktury (vztahy), kolekce
- Speciální typy reprezentující volání API (*root types*)
  - Query – čtení dat
  - Mutation – změna dat
- GraphQL Schema Definition Language (SDL)

GraphQL – definice typů

```
type Person {
  name: String!
  age: Int!
  posts: [Post!]!
}
```

```
type Car {
  type: String!
  reg: String!
  owner: Person!
}
```

```
type Query {
  allPersons: [Person!]!
  findPerson(name: String!): Person!
}
```

```
type Mutation {
  createPerson(name: String!, age: Int!): Person!
}
```

GraphQL – dotazy

```
{
  allPersons {
    name
  }
}
```

```
{
  "allPersons": [
    { "name": "Jan" },
    { "name": "Karolína" },
    { "name": "Alice" }
  ]
}
```

```
{
  findPerson(name: "James") {
    name
    age
    cars {
      type
    }
  }
}
```

```
{
  "findPerson": {
    "name": "James",
    "age": 28,
    "cars": [
        { "type": "Fiat" },
        { "type": "Tesla" }
    ]
  }
}
```

GraphQL – modifikace

```
mutation AddPerson($person: PersonInput)
    updatePerson(p: $person) {
        id
    }
}
```

```
 {
    "person": {
       "born": "1991-06-03T22:00:00Z[UTC]",
       "id": 1154731299,
       "name": "Sylvester",
       "surname": "Stalone"
    }
}
```

```
{
    "data": {
        "updatePerson": {
            "id": 1154731299
        }
    }
}
```

GraphQL přes HTTP

- Jediné endpoint URL
- Odeslání přes GET

  `http://myapi/graphql?query={me{name}}`
- Odeslání přes POST
  - Data `application/json`

    `{"query": "{me{name}}"}`

    `{"query": "{mutation … }", "variables": {name: "Jan"}}`
  - Data `application/graphql`

    `{me{name}}`

Implementace GraphQL

- Klientská strana
  - Žádná speciální podpora – posílání POST požadavků
- Serverová strana
  - Definice datových struktur a dotazů
  - Zpracování SDL nebo generování SDL z kódu
  - Mnoho knihoven
    https://graphql.org/code/

Microprofile GraphQL

- Demo: Open Liberty Guide, demo aplikace
- Závislost v `pom.xml`

```
<dependency>
    <groupId>org.eclipse.microprofile.graphql</groupId>
    <artifactId>microprofile-graphql-api</artifactId>
    <version>2.0</version>
    <scope>provided</scope>
</dependency>
```

- V Open Liberty přidat vlastnost na server

```
<feature>mpGraphQL-2.0</feature>
```

Definice API

```
@GraphQLApi
@RequestScoped
public class Api
{
    @Inject
    private PersonManager personMgr;

    @Query
    @Description("Gets the complete list of people")
    public List<Person> getPeople()
    {
        return personMgr.findAll();
```

Definice datových typů

- Standardní java třídy (POJO)
- Volitelně lze nastavit mapování
  - jmen tříd – `@Type("…")`
  - vlastností – `@Name("…")`
  - a doplnit popis významu – `@Description`.

Autentizace v GraphQL

- Stejný mechanismus, jako u REST
- Stejná konfigurace JWT

```
@GraphQLApi
@LoginConfig(authMethod = "MP-JWT", realmName = "MP-JWT")
@RequestScoped
public class Api
{
    @Inject
    private PersonManager personMgr;

    @Query
    @RolesAllowed("admin")
    @Description("Gets the complete list of people")
    public List<Person> getPeople()
```

Správa identit

OAuth2, OpenID Connect, SAML

Správa identit v IS

- Kdo je uživatel a jaká má práva?
  - **Autentizace** – ověření identity (kdo to je?)
  - **Autorizace** – ověření oprávnění (co smí dělat?)
- Problémy centralizované správy
  - Každá aplikace spravuje vlastní uživatele → duplicita, bezpečnostní riziko
  - Uživatel si pamatuje mnoho hesel
- Řešení: **federovaná identita**
  - Identita je spravována centrálně, aplikace ji sdílejí
  - **Single Sign-On (SSO)** – přihlásíte se jednou, funguje všude

Základní koncepty

- **Identity Provider (IdP)** – server, který spravuje identity a vydává tokeny
  - Příklady: Google, Facebook, Microsoft Entra ID, Keycloak
- **Service Provider / Relying Party** – aplikace, která spoléhá na IdP
- **Token** – doklad o prokázané identitě nebo oprávnění
  - Přenášen mezi stranami, ověřitelný bez dotazu na IdP
- **Claim** – tvrzení o uživateli obsažené v tokenu
  - Např. jméno, e-mail, role, organizace

Přehled standardů

Standard	Účel	Vrstva
OAuth 2.0	Delegovaná autorizace	Přístup ke zdrojům
OpenID Connect (OIDC)	Autentizace nad OAuth2	Identita uživatele
SAML 2.0	Autentizace + autorizace	SSO v podnicích
JWT	Formát tokenu	Datový nosič

OAuth 2.0

- https://oauth.net/2/
- **Delegovaná autorizace** – aplikace získá omezený přístup ke zdrojům jménem uživatele
  - „Přihlásit přes Google" – Google vydá token, aplikace ho použije
- Uživatel nikdy nesděluje heslo třetí aplikaci
- Definuje **role**:
  - **Resource Owner** – uživatel vlastnící data
  - **Client** – aplikace žádající o přístup
  - **Authorization Server** – vydává tokeny
  - **Resource Server** – API chráněné tokeny

OAuth 2.0 – typy tokenů

- **Access Token** – krátkodobý, opravňuje k přístupu na Resource Server
- **Refresh Token** – dlouhodobý, slouží k získání nového Access Tokenu bez přihlášení
- **Authorization Code** – dočasný kód vyměněný za tokeny (přenášen přes prohlížeč)
- Doporučený formát: **JWT** (ale OAuth 2.0 formát nestandardizuje)

OAuth 2.0 – Authorization Code Flow

```
Uživatel              Klient                Auth Server          Resource Server
   |                    |                     |                       |
   |-- přihlásit se -->|                     |                       |
   |                    |-- redirect -------->|                       |
   |<------- login page -------------------- |                       |
   |-------- přihlásí se + souhlas --------->|                       |
   |<------- redirect + auth_code -----------|                       |
   |                    |<-- auth_code ------|                       |
   |                    |-- code + secret -->|                       |
   |                    |<-- access_token ---|                       |
   |                    |-- access_token ---------------------------->|
   |                    |<-- data ------------------------------------|
```

OAuth 2.0 – Grant typy

- **Authorization Code** – pro webové a mobilní aplikace (nejbezpečnější)
  - S rozšířením **PKCE** (Proof Key for Code Exchange) pro veřejné klienty
- **Client Credentials** – pro komunikaci server-server bez uživatele
- **Device Code** – pro zařízení bez prohlížeče (SmartTV, IoT)
- ~~Implicit~~ – zastaralý, nahrazen Authorization Code + PKCE
- ~~Resource Owner Password~~ – zastaralý, heslo putuje ke klientovi

OAuth 2.0 – Scopes

- Rozsah oprávnění, o která aplikace žádá
- Definuje Resource Server, uživatel musí souhlasit
- Příklady (Google):
  - `openid` – základní identita
  - `email` – přístup k e-mailové adrese
  - `https://www.googleapis.com/auth/calendar` – kalendář
- Princip nejmenšího oprávnění – žádáme jen co potřebujeme

OpenID Connect (OIDC)

- https://openid.net/connect/
- **Tenká autentizační vrstva nad OAuth 2.0**
  - OAuth 2.0 řeší autorizaci, OIDC přidává **autentizaci**
  - Klient se dozví *kdo* je uživatel, ne jen že má přístup
    - Např. jméno, e-mail, fotka
- Přidává:
  - **ID Token** – JWT s informacemi o uživateli (identity claims)
  - **UserInfo Endpoint** – dotaz na další claims
  - Standardizované claims: `sub`, `name`, `email`, `picture`, …
  - **Discovery Dokument** – popis IdP na `/.well-known/openid-configuration`

OIDC – ID Token

- JWT vydaný Auth Serverem po úspěšném přihlášení
- Standardní claims:
  - `iss` – vydavatel (IdP URL)
  - `sub` – unikátní identifikátor uživatele u IdP
  - `aud` – příjemce (Client ID aplikace)
  - `exp`, `iat` – expirace a čas vydání

```
{
  "iss": "https://accounts.google.com",
  "sub": "110169484474386276334",
  "aud": "812741506391",
  "email": "jan.novak@gmail.com",
  "name": "Jan Novák",
```

OIDC – tok přihlášení

1. Klient přesměruje uživatele na IdP s parametry:
   - `response_type=code`, `scope=openid email`, `client_id`, `redirect_uri`, `nonce`
2. Uživatel se přihlásí u IdP
3. IdP přesměruje zpět s `code`
4. Klient vymění `code` za `access_token` + `id_token`
5. Klient ověří podpis a claims v `id_token`
6. Volitelně: dotaz na `userinfo` endpoint pro další data

PKCE – Proof Key for Code Exchange

- Rozšíření Authorization Code Flow pro **veřejné klienty**
  - Mobilní aplikace, SPA (nemohou bezpečně uchovat client_secret)
- Princip:

1. Klient vygeneruje náhodný `code_verifier`
2. Odešle hash `code_challenge = SHA256(code_verifier)` s požadavkem
3. Auth Server si uloží `code_challenge`
4. Při výměně kódu klient pošle `code_verifier`
5. Server ověří: `SHA256(code_verifier) == code_challenge`

- Chrání před zachycením `authorization_code`

SAML 2.0

- https://www.oasis-open.org/committees/security/
- **Security Assertion Markup Language**
- Starší standard (2005), XML-based, rozšířený v podnicích
- Zaměřen na **SSO** mezi organizacemi (federace)
  - Např. přihlášení přes firemní IdP do externích SaaS aplikací
- Role:
  - **Identity Provider (IdP)** – spravuje uživatele, vydává assertions
  - **Service Provider (SP)** – aplikace, přijímá assertions

SAML 2.0 – tok přihlášení (SP-initiated)

1. Uživatel přistoupí na SP (aplikaci)
2. SP přesměruje na IdP s `AuthnRequest`
3. Uživatel se přihlásí u IdP
4. IdP pošle **SAML Response** s podepsanou Assertion na SP (přes prohlížeč, POST)
5. SP ověří podpis a zpracuje assertions
6. Uživatel je přihlášen

- Assertion je podepsána privátním klíčem IdP, SP ověřuje veřejným klíčem

Keycloak

- https://www.keycloak.org/
- Open-source **Identity and Access Management** server
- Implementuje OAuth 2.0, OIDC, SAML 2.0
- Správa uživatelů, rolí, skupin
- Federace s LDAP/Active Directory
- Sociální přihlášení (Google, GitHub, ...)
- Administrátorská konzole, REST API pro správu
- Alternativy: Auth0, Microsoft Entra ID, Okta, AWS Cognito

OAuth2 / OIDC v Javě – MicroProfile JWT

- Ověření JWT access tokenů na straně Resource Serveru
- Neřeší samotný tok OAuth2 (přihlášení, výměna tokenů)
- Konfigurace ověření tokenu:

```
mp.jwt.verify.publickey.location=http://keycloak:8080/realms/myrealm/protoc
mp.jwt.verify.issuer=http://keycloak:8080/realms/myrealm
mp.jwt.verify.audiences=my-client-id
```

- Roles z JWT claim `groups` nebo `realm_access.roles` (záleží na mapování)

OAuth2 / OIDC v Javě – Jakarta Security 3.0

- Standard Jakarta EE 10
- `@OpenIdAuthenticationMechanismDefinition` – konfigurace OIDC klienta

```
@OpenIdAuthenticationMechanismDefinition(
    providerURI = "http://keycloak:8080/realms/myrealm",
    clientId = "my-app",
    clientSecret = "${oidcConfig.clientSecret}",
    redirectURI = "${baseURL}/callback",
    jwksConnectTimeout = 5000
)
@ApplicationScoped
public class AppConfig { }
```

- Zvládá celý Authorization Code Flow automaticky
- Dostupné na Payara 6+, WildFly 27+

Správa identit v Open Liberty

- Open Liberty podporuje všechny hlavní standardy přes **features**

Feature	Účel
`openidConnectClient-1.0`	OIDC klient (Relying Party)
`openidConnectServer-1.0`	OIDC provider (IdP)
`socialLogin-1.0`	OAuth2/OIDC sociální přihlášení
`samlWeb-2.0`	SAML 2.0 Web SSO
`mpJwt-2.1`	MicroProfile JWT (ověření tokenů)
`appSecurity-5.0`	Jakarta Security 3.0 (OIDC anotace)

- Dokumentace: https://openliberty.io/docs/latest/single-sign-on.html

Open Liberty – OIDC klient

- Feature `openidConnectClient-1.0` – Liberty jako OIDC Relying Party
- Konfigurace v `server.xml`, zvládá celý Authorization Code Flow

```
<featureManager>
    <feature>openidConnectClient-1.0</feature>
</featureManager>

<openidConnectClient
    id="myOIDC"
    clientId="my-app"
    clientSecret="secret"
    discoveryEndpointUrl=
      "http://keycloak:8080/realms/myrealm/.well-known/openid-configuration"
    redirectToRPHostAndPort="https://my-app.example.com"
    scope="openid profile email"
    userNameAttribute="preferred_username"
    groupIdentifier="groups" />
```

- Guide: https://openliberty.io/docs/latest/enable-openid-connect-client.html

Open Liberty – SAML

- Feature `samlWeb-2.0` – Liberty jako SAML Service Provider
- Konfigurace v `server.xml`:

```
<featureManager>
    <feature>samlWeb-2.0</feature>
</featureManager>

<samlWebSso20 id="defaultSP"
    signatureMethodAlgorithm="SHA256"
    allowCustomCacheKey="false">
    <pkixTrustEngine trustAnchor="samlTrustStore" />
</samlWebSso20>

<keyStore id="samlTrustStore"
    location="saml-truststore.jks"
    type="JKS"
    password="changeit" />
```

- Metadata SP dostupná na `/ibm/saml20/{id}/samlmetadata`

Open Liberty – Social Login

- Feature `socialLogin-1.0` – přihlášení přes OAuth2/OIDC providery
- Vestavěná podpora: GitHub, Google, Facebook, LinkedIn, Twitter
- Vlastní OAuth2 nebo OIDC provider:

```
<featureManager>
    <feature>socialLogin-1.0</feature>
</featureManager>

<oidcLogin id="keycloak"
    clientId="my-app"
    clientSecret="secret"
    discoveryEndpoint=
      "http://keycloak:8080/realms/myrealm/.well-known/openid-configuration"
    userNameAttribute="preferred_username" />

<!– nebo pro čistý OAuth2: –>
<oauth2Login id="myProvider"
    clientId="…" clientSecret="…"
```

Open Liberty – Jakarta Security (OIDC)

- Feature `appSecurity-5.0` – Jakarta Security 3.0
- Konfigurace výhradně anotacemi, **bez změn v `server.xml`**

```
<featureManager>
    <feature>appSecurity-5.0</feature>
</featureManager>
```

- Guide + Keycloak: https://openliberty.io/blog/2024/07/31/keycloak-with-openliberty.html

Open Liberty – MicroProfile JWT

- Feature `mpJwt-2.1` – ověření JWT Bearer tokenů na Resource Serveru
- Konfigurace v `server.xml` nebo `microprofile-config.properties`
- Přístup k tokenu v kódu:

```
@Inject
JsonWebToken jwt;

String user  = jwt.getSubject();
Set<String> roles = jwt.getGroups();
```

- Guide: https://openliberty.io/guides/microprofile-jwt.html

Přístup k identitě v Javě

```java
@Inject
private OpenIdContext openIdContext;

@GET
@Path("/profile")
@RolesAllowed("user")
public String getProfile() {
    String subject = openIdContext.getSubject();
    Optional<String> email = openIdContext.getClaims()
        .getStringClaim("email");
    return "Přihlášen: " + email.orElse(subject);
}
```

Refresh Token – princip

- **Access Token** je záměrně krátkodobý (minuty) – omezuje škody při úniku
- **Refresh Token** je dlouhodobý (hodiny–dny) a slouží k obnovení Access Tokenu **bez opětovného přihlášení uživatele**
- Refresh Token je uložen bezpečně u klienta, **nikdy se neposílá na Resource Server**

Refresh Token – flow

```
Klient                Auth Server         Resource Server
  |                      |                      |
  |-- access_token ------------------------->|
  |<-- 401 Unauthorized -------------------- |   (token expiroval)
  |                      |                      |
  |-- refresh_token -->|                      |
  |<-- nový access_token + (nový refresh_token)
  |                      |                      |
  |-- nový access_token -------------------->|
  |<-- data ------------------------------- |
```

Refresh Token – bezpečnost

- **Refresh Token Rotation** – každým použitím se refresh token vyměňuje za nový
  - Starý token je okamžitě invalidován
  - Detekce opětovného použití (*reuse detection*): pokud někdo použije starý token, celá rodina tokenů se revokuje
- **Revokace** – refresh token lze explicitně zneplatnit (odhlášení, změna hesla)
  - RFC 7009: Token Revocation Endpoint
- Refresh Token **neobsahuje** data uživatele – je to neprůhledný (opaque) identifikátor nebo JWT s minimálními claims

Doporučené postupy

- Nikdy neukládejte tokeny do `localStorage` – použijte `HttpOnly` cookies nebo server-side session
- Vždy ověřujte podpis tokenu a claim `iss`, `aud`, `exp`
- Používejte krátkou expiraci Access Tokenu (minuty), delší Refresh Tokenu
- **PKCE** pro všechny veřejné klienty
- Minimalizujte scopes – žádejte jen nezbytné
- Rotujte Refresh Tokeny (*refresh token rotation*)
- HTTPS všude – tokeny jsou citlivé přihlašovací údaje

A to je vše!

Dotazy?